ANALYSIS OF ALGORITHM TIME COMPLEXITY

Problem 1: Analyzing Complexity with the Counter Method

```
#include <stdio.h>

int main() {

int n;

if (scanf("%d", &n) != 1) return 0;

int count = 0;

int i = 1;

count++;

int s = 1;

count++;

while (++count && s <= n) {

i++;

count++;

s += i;

count++;

}

printf("%d\n", count);

return 0
```

```
}
```

Problem 2: Analyzing Complexity with the Counter Method

```
#include <stdio.h>

void func(int n) {
    int count = 0;
     count++;
    if(n == 1) {
       count++;
    }
    else {
       for(int i = 1; i <= n; i++) {
          count++;
      for(int j = 1; j <= n; j++) {
            count++;
                  count++;
            count++;
```

```
            count++;
            break;
        }
    }
    count++;
  }

  printf("%d\n", count);
}

int main() {
  int n;
  if (scanf("%d", &n) == 1 && n > 0) {
    func(n);
  }
```

```
    return 0;


}
```

Problem 3: Analyzing Complexity with the Counter Method

```
#include <stdio.h>

int main()

{

int n, i;

int count = 0;

scanf("%d", &n);

for (i = 1; i <= n; ++i)

{

count++;

if (n % i == 0)

{

count++;

}

count++;

}

count++;

printf("%d", count);

return 0;
```

```
}
```

Problem 4: Analyzing Complexity with the Counter Method

```
#include <stdio.h>

void function(int n)

{

int c = 0;

int i = n / 2;

c++;

for (; (c++, i < n); i++)

{

for (int j = 1; (c++, j < n); j = 2 * j)

{

for (int k = 1; (c++, k < n); k = k * 2)

{

c++;

}

}

}

printf("%d", c);
```

```
}

int main()

{

int n;

scanf("%d", &n);

function(n);

return 0;

}
```

Problem 5: Analyzing Complexity with the Counter Method

```
#include <stdio.h>

void reverse(int n)

{

int rev = 0, remainder;

int c = 2;

while (n != 0)

{

remainder = n % 10;

c++;

rev = rev * 10 + remainder;
```

```
c++;
n /= 10;
c++;
c++;
}
c++;
printf("%d", c);
}
int main()
{
int n;
scanf("%d", &n);
reverse(n);
return 0;
}
```